

NAIVE BAYES

CARRERA: TECNICO SUPERIOR EN SISTEMAS INFORMATICOS.

MAESTRA: EMILIO BARAJAS GONZALEZ.

MATERIA: APRENDIZAJE MAQUINA.

INTEGRANTES:

- Angelica Yaneth Carrillo Banda, 221822469
- Citlaly Lizeth Cruz Gomez, 221821756.
- Juan Pablo Angel Jimenez Palos, 221821993.
- Ricardo Rodriguez Jaramillo, 221821829.



01

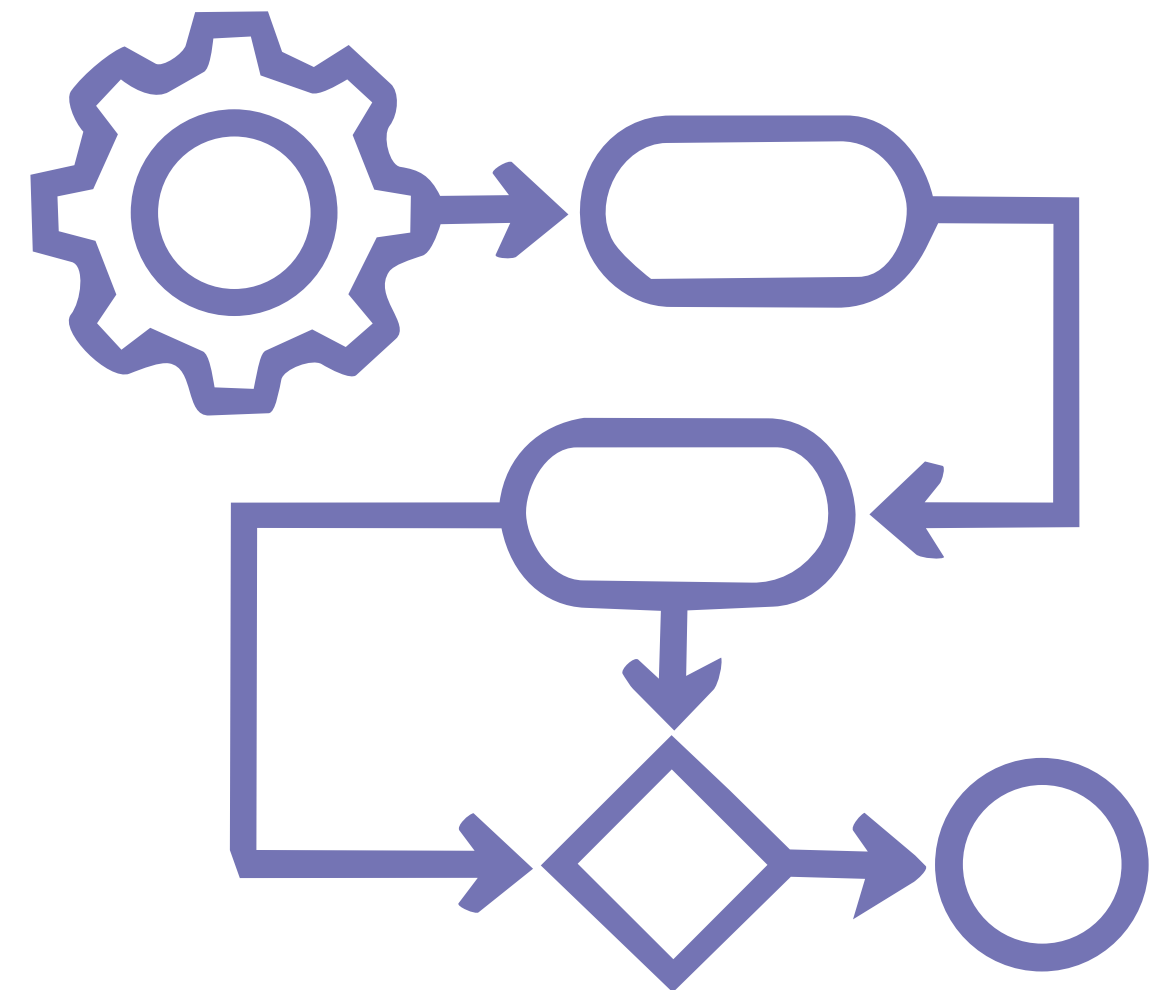
INTRODUCCION

Algoritmos: Metodos basicos.

Dada la infinita variedad de dataset y sus diferentes estructuras, es fundamental adoptar una metodología que priorice la **simplicidad**.

Objetivo

El objetivo es **elegir el algoritmo adecuado** para la estructura de los datos; de lo contrario, existe la posibilidad de obtener modelos complejos y confusos.



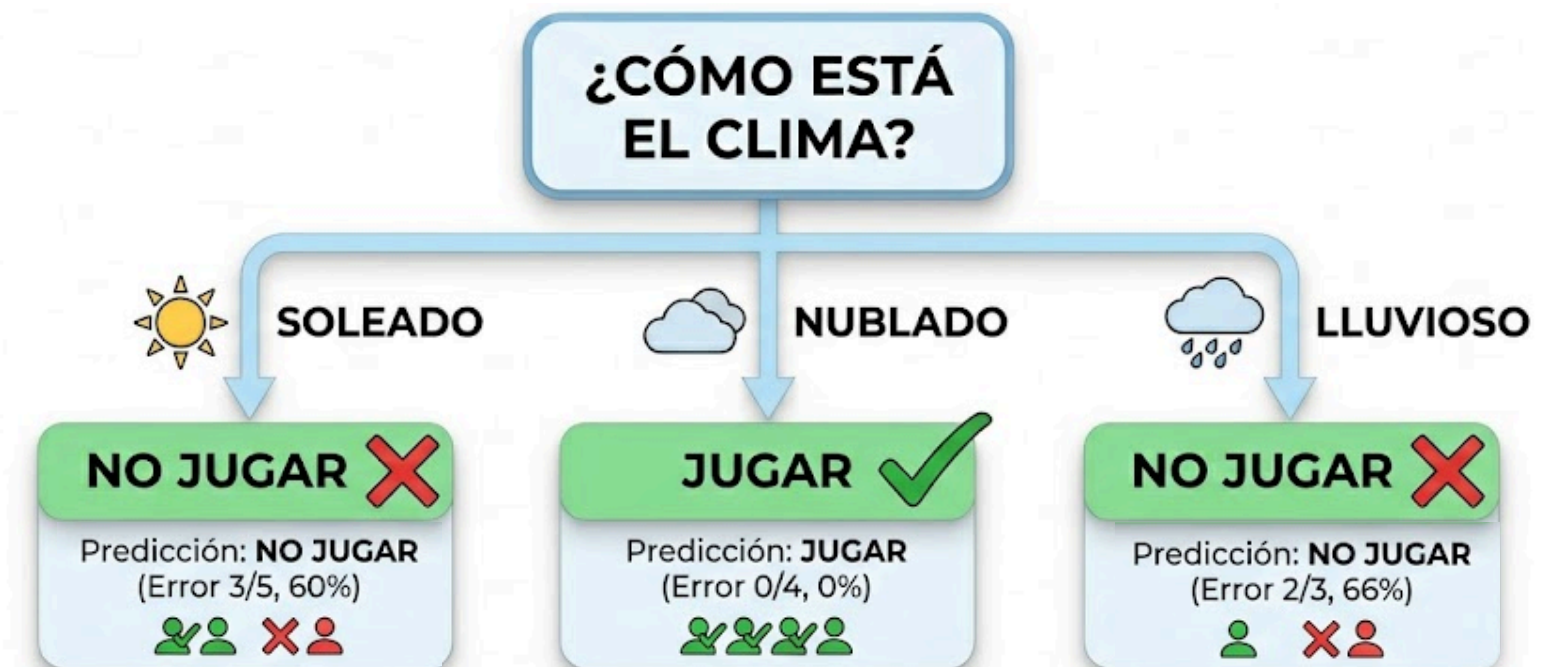
02

**REGLAS
RUDIMENTARIAS**

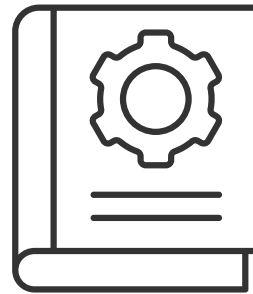
1R: 1-rule.

Reglas de clasificación simple.

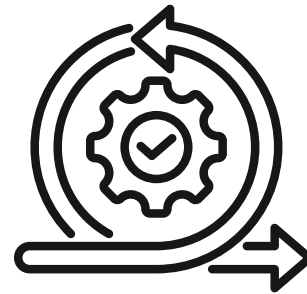
- Genera un **arbol de decision** de un nivel.
 - Produce reglas por atributo individual para clasificar la estructura de los datos.
 - Cada atributo genera un conjunto diferente de reglas.
 - Evalua todas las reglas generadas.
 - Selecciona el atributo con menor cantidad de errores.



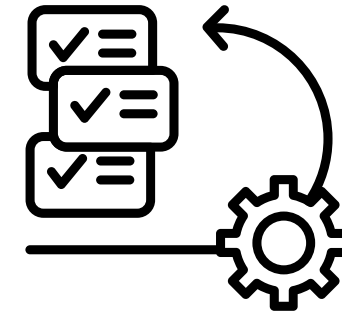
Ventajas principales



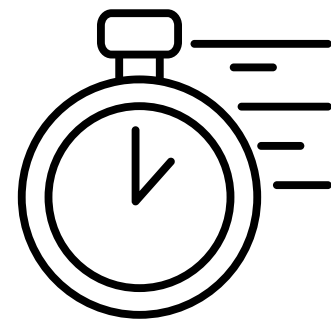
Linea base/primer
paso.



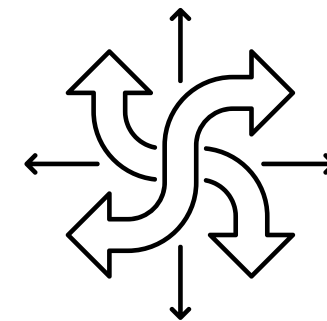
Preciso y
eficiente.



Mejor
interpretabilidad



Velocidad y bajo costo
computacional.



Manejo versátil de
datos

¿Y ante errores?

A pesar de ser un algoritmo rudimentario, 1R tiene su manera de afrontar valores faltantes y atributos numericos.

Valores faltantes.

- Si existe un valor nulo, el algoritmo lo trata simplemente como una categoría adicional.
 - Por ejemplo, el atributo clima pasaría a tener cuatro posibles valores: soleado, nublado, lluvioso y 'nulo'.

Atributos numericos.

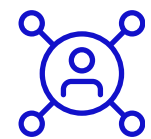
- Se transforman en categorías (discretización) agrupándolos en rangos.
- Busca los mejores "puntos de corte" numéricos para separar las clases
- Para no crear reglas inútiles, exige un mínimo de ejemplos de la misma clase por cada rango creado

03

MODELO ESTADISTICO

Naïve Bayes

Modelado Estadístico en Machine Learning



Basado en Bayes

Usa la regla de
probabilidad condicional



Independencia

Asume atributos independientes
entre sí



Simple y Rápido

Funciona sorprendentemente
bien en práctica



Versátil

Clasifica texto, datos
numéricos y más

¿Cómo funciona Naïve Bayes?

- 1

Contar frecuencias por clase
- 2

Calcular probabilidades
- 3

Multiplicar y normalizar

Tabla de Probabilidades — Datos del Clima

Atributo	Valor	Sí (play=yes)	No (play=no)	P(val Sí)	P(val No)
Outlook	Sunny	2	3	2/9/2026	3/5/2026
	Overcast	4	0	4/9/2026	0/5
	Rainy	3	2	3/9/2026	2/5/2026
Humidity	High	3	4	3/9/2026	4/5/2026
	Normal	6	1	6/9/2026	1/5/2026
Windy	VERDADERO	3	3	3/9/2026	3/5/2026
Total días	—	9	5	9/14	5/14

Statistical Modeling

Nuevo día: Sunny, Cool, High, True

$$P(\text{Sí}) = \frac{2}{9} \times \frac{3}{9} \times \frac{3}{9} \times \frac{9}{14}$$
$$= 0.0053$$

$$P(\text{No}) = \frac{3}{5} \times \frac{4}{5} \times \frac{3}{5} \times \frac{5}{14}$$
$$= 0.0206$$

Resultado: NO jugar
20.5% Sí | 79.5% No

Problema de Probabilidad Cero y Estimador de Laplace

El Problema

Si un valor de atributo NUNCA aparece con cierta clase en el entrenamiento, su probabilidad = 0

Esto anula todas las demás probabilidades al multiplicar, sin importar cuánta evidencia haya en contra

Solución: Estimador de Laplace

Suma 1 a cada conteo y ajusta el denominador:

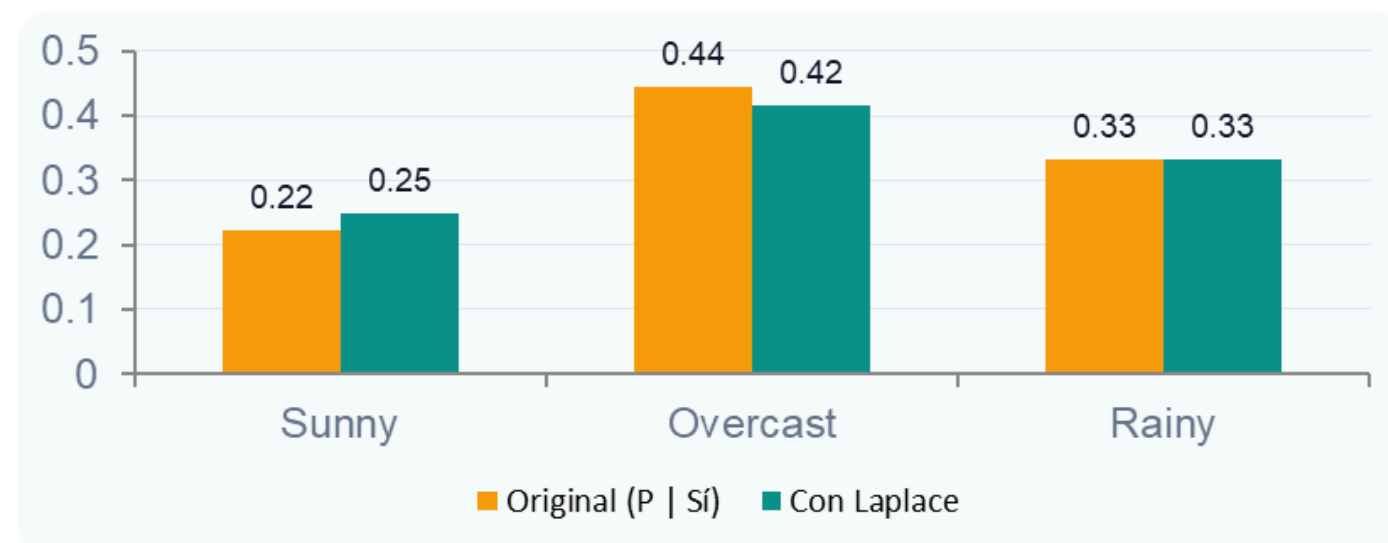
Original: $2/9$, $4/9$, $3/9$

Con Laplace:

$(2+1)/(9+3)$, $(4+1)/(9+3)$, $(3+1)/(9+3)$

= $3/12$, $5/12$, $4/12$

Comparación: Probabilidades Originales vs. con Estimador de Laplace (Outlook | Sí)



Laplace garantiza que ningún valor tenga probabilidad exactamente 0

Un μ grande da más peso a probabilidades a priori.

Un μ pequeño da más importancia a los datos de entrenamiento.

Atributos Numéricos, Valores Faltantes y Limitaciones

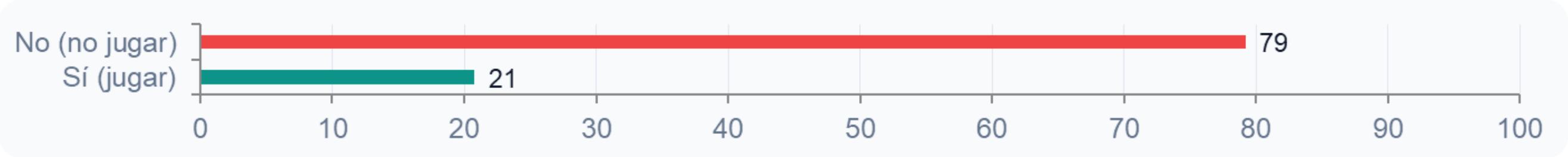
Atributos Numéricos: Distribución Normal

Atributo	Clase	Media (μ)	Desv. Est. (σ)
Temperatura	Sí	73	6.2
	No	74.6	7.9
Humedad	Sí	79.1	10.2
	No	86.2	9.7

$$f(x \mid \mu, \sigma) = (1 / \sigma \sqrt{2\pi}) \times e^{-(x-\mu)^2 / 2\sigma^2}$$

Ejemplo: $f(\text{temp}=66 \mid \text{Sí}) = 0.034 \rightarrow f(\text{hum}=90 \mid \text{Sí}) = 0.022$

Resultado para Nuevo Día (Sunny, 66°F, 90% Hum, Windy)



Limitaciones Principales

 Atributos redundantes

Si un atributo se repite, su peso se eleva al cuadrado, distorsionando el resultado.

 Asume distribución normal

No siempre es correcta. Para otras distribuciones se usan estimadores de densidad.

 Ignora dependencias

Atributos dependientes reducen el poder de clasificación del modelo.

 Valores faltantes: sin problema

Simplemente se omiten del cálculo; no afectan el resultado.

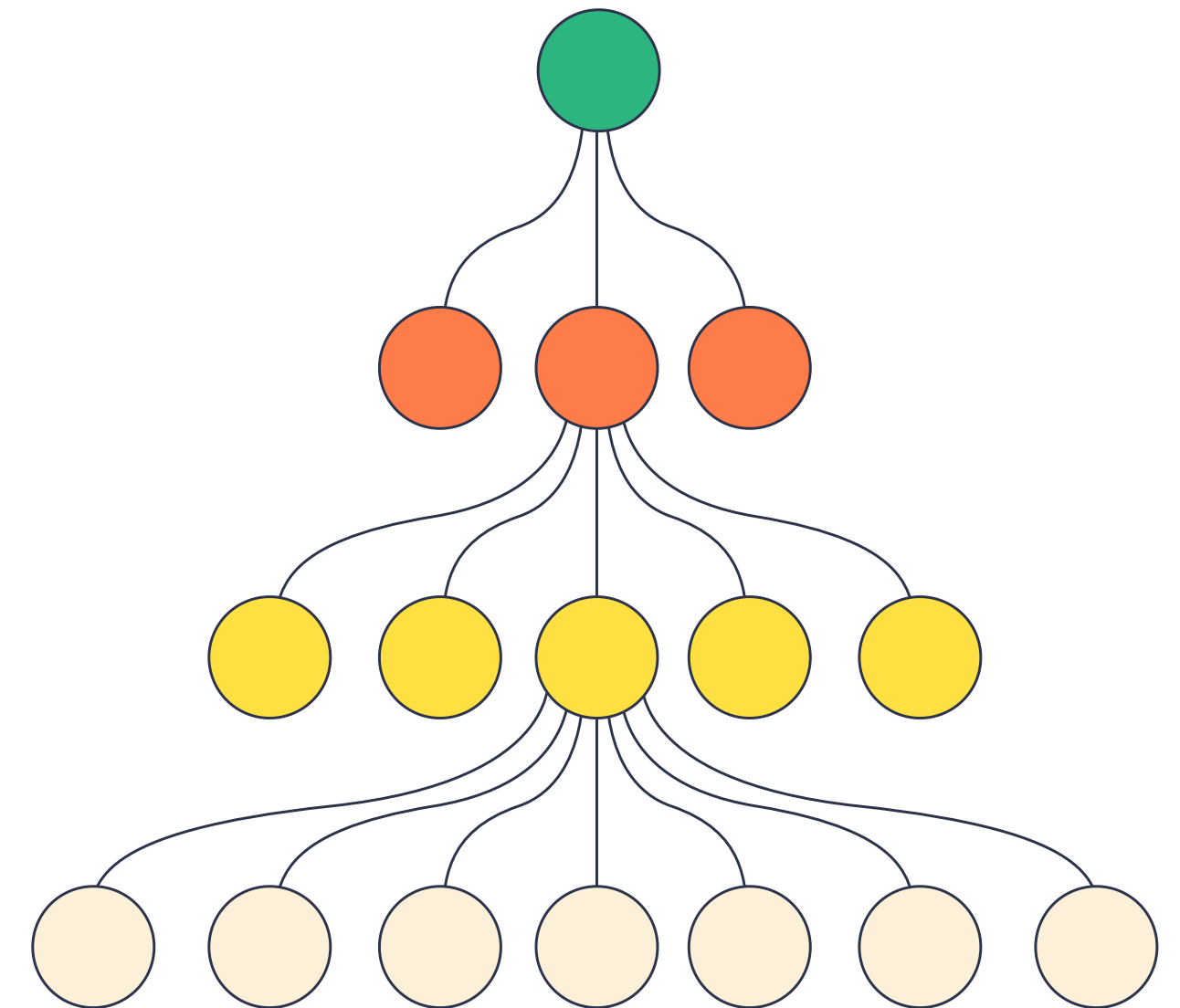
04

DIVIDE Y CONQUISTA

Construcción de Árboles de Decisión

- Primero, se selecciona un atributo para colocarlo en el nodo raíz y se crean subconjuntos y una rama por cada uno.
- Se crea una rama para cada valor posible. Esto divide el conjunto de ejemplos en subconjuntos, uno por cada valor del atributo.
- Ahora, el proceso puede repetirse para cada rama.
- Si en cualquier momento todas las instancias de un nodo tienen la misma clasificación, se detiene el desarrollo de esa parte del árbol.

Divide y conquista



La medida de la información

La medida de **pureza** que se maneja se llama información, que se mide en **bits** (mayormente menor a 1)

Se necesita para calcular la incertidumbre de la información (entropía o desorden)

- **Pureza Máxima:** Si el nodo es 100% una sola clase, la información es 0.
- **Incertidumbre Total:** Si las clases están equilibradas (50/50), la información es máxima (1 bit).

0-1

Calculando la información

Para que una fórmula sea útil en un árbol de decisión, debe poder tener estos aspectos:

Si no hay duda, el valor es cero

- Si todos los ejemplos son "verdaderos", no necesitas más información. El valor de información debe ser 0.

Si hay máxima duda, es máxima

- Si la mitad de los valores son "Verdaderos" y la otra mitad son "Falsos" la entropía sube a 1

Deberá ser para múltiples casos

- No solo se debe a cerrar a 1 y 0 si no a valores intermedios dependiendo de la entropía.

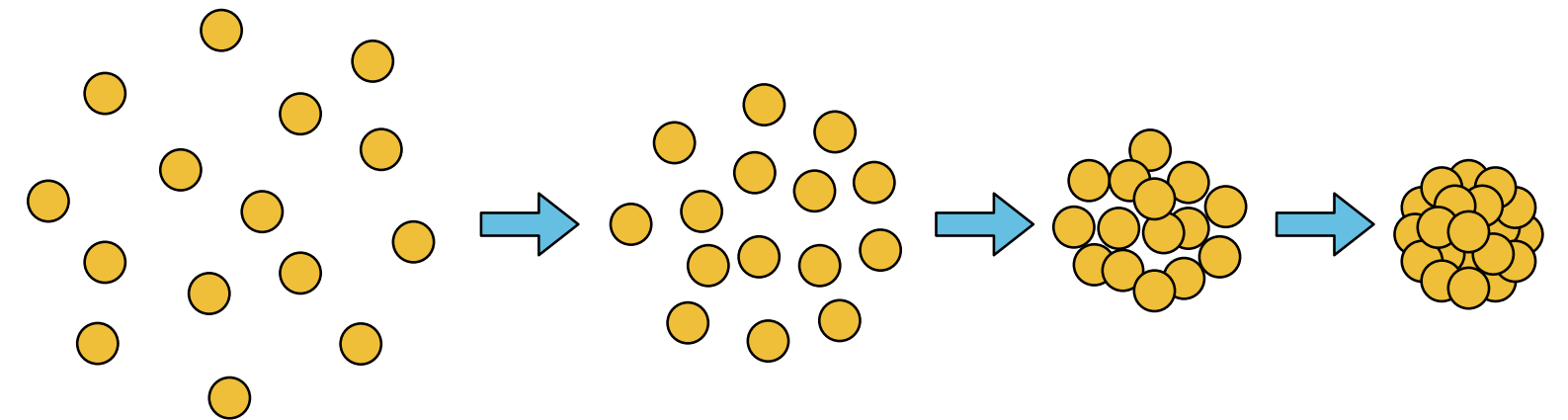
Calculando la información

Entropía

Para que todas esas métricas se puedan cumplir se usa la formula de la **entropía**

Entropía ($p_1, p_2 \dots p$) = $-p_1 \log p_1 - p_2 \log p_2$
.... $-p \log p$

¿Por qué el signo negativo? Porque como (p) son fracciones (como 0.5), sus logaritmos son negativos. Se usa para obtener positivos.

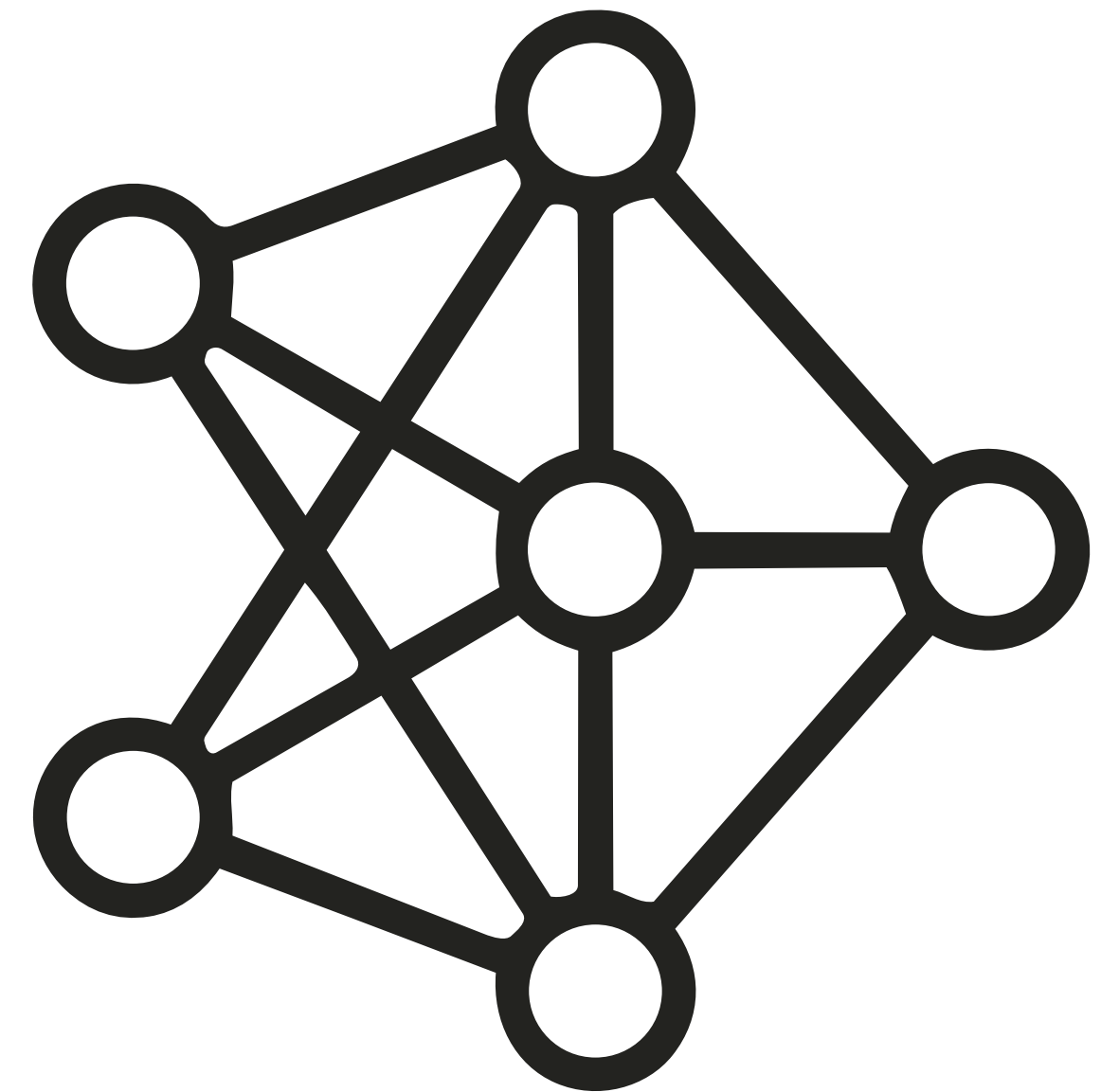


Atributos altamente ramificados

Cuando algunos atributos tienen una **gran cantidad de valores posibles**, dando lugar a una rama de múltiples vías con **muchos nodos hijos**, surge un problema con el cálculo de la ganancia de información.

Tiende a elegir atributos que tienen muchísimos valores diferentes (como un código de ID).

Esto es un error porque esos atributos memorizan los datos en lugar de aprender patrones útiles para el futuro.



Atributos altamente ramificados

Para solucionar esto se aplica una especie de multa a través de la razón de la ganancia o Gain Ratio.

Es la solución matemática que penaliza a los atributos que se **ramifican demasiado**. Se calcula dividiendo la ganancia original entre la "información intrínseca" (qué tan compleja es la división).

$$\textit{Gain ratio} = \frac{\textit{Gain}(A)}{\textit{SplitInfo}(A)}$$

05

ALGORITMOS DE COBERTURA

¿Qué son los Covering Algorithms?

Son una **técnica de clasificación** que **construye reglas** directamente a partir de los datos. En lugar de generar estructuras como árboles de decisión, este enfoque **crea condiciones** que permiten identificar a qué **clase pertenece** cada ejemplo.

Cada regla busca “**cubrir**” un **conjunto de instancias** que **pertenecen a la misma clase**.

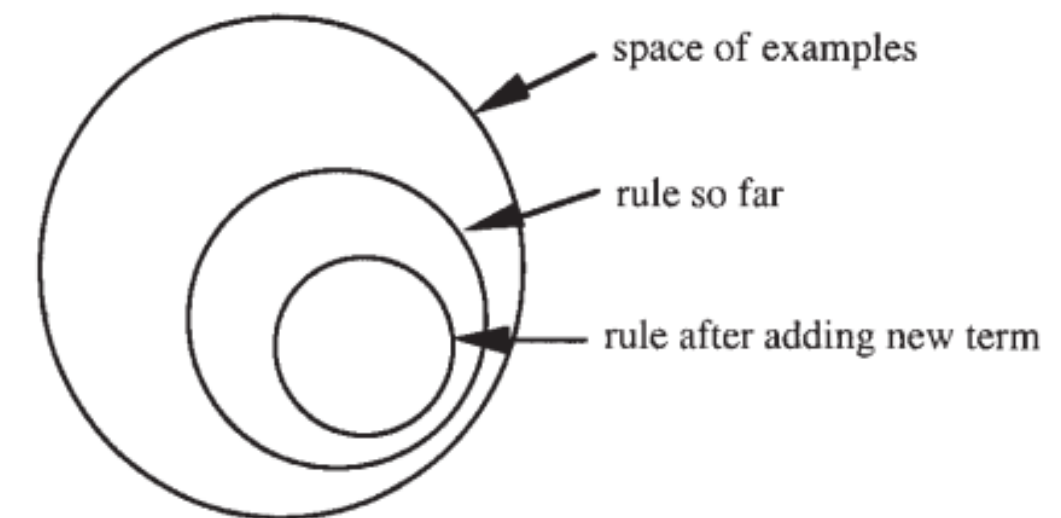
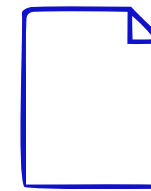


Figure 4.7 The instance space during operation of a covering algorithm.

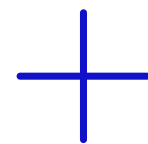
Idea general del funcionamiento



Se analiza una clase a la vez



Se inicia con una regla vacía



Se agregan condiciones
progresivamente



Cada condición mejora
la precisión



Se guarda la regla cuando es
suficientemente buena

Construcción de una regla

El algoritmo comienza con una **regla muy general** y la va **refinando** poco a poco **agregando condiciones**.

Por ejemplo, puede iniciar con:

“If $x > 1.2$ → clase A”

Pero como aún hay errores, se agrega otra condición:

“If $x > 1.2$ AND $y > 2.6$ → clase A”

Cada nueva condición **reduce los errores** y hace la regla **más precisa**.



Diferencia con árboles de decisión

Covering:

- Genera reglas directamente
- Se enfoca en una clase por vez

Árboles de decisión:

- Dividen los datos en forma jerárquica
- Consideran todas las clases simultáneamente

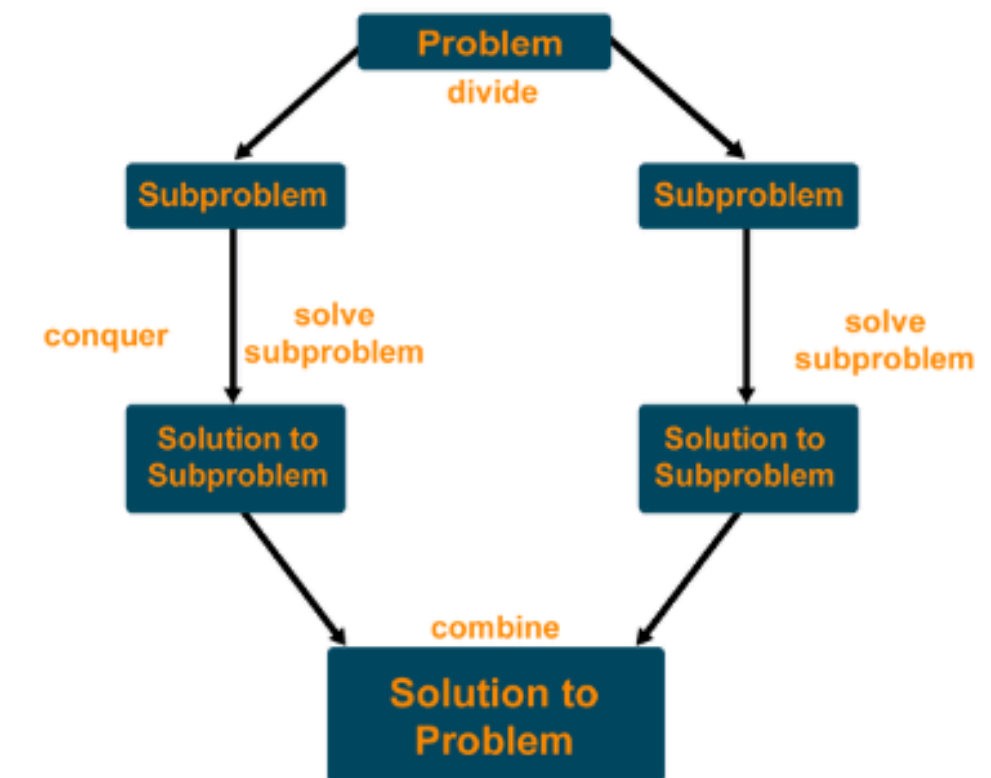
- Ambos pueden representar lo mismo, pero de manera distinta

Lógica del algoritmo

El proceso consiste en **generar reglas** para **cada clase** y, una vez que una regla **cubre ciertos datos**, estos se **eliminan del conjunto**. Después, el **algoritmo continúa** con los datos restantes hasta **cubrir todos los casos**.

Este enfoque se conoce como **separate-and-conquer**, ya que **divide el problema** en partes más pequeñas **resolviendo una regla a la vez**.

Divide and Conquer Algorithm



06

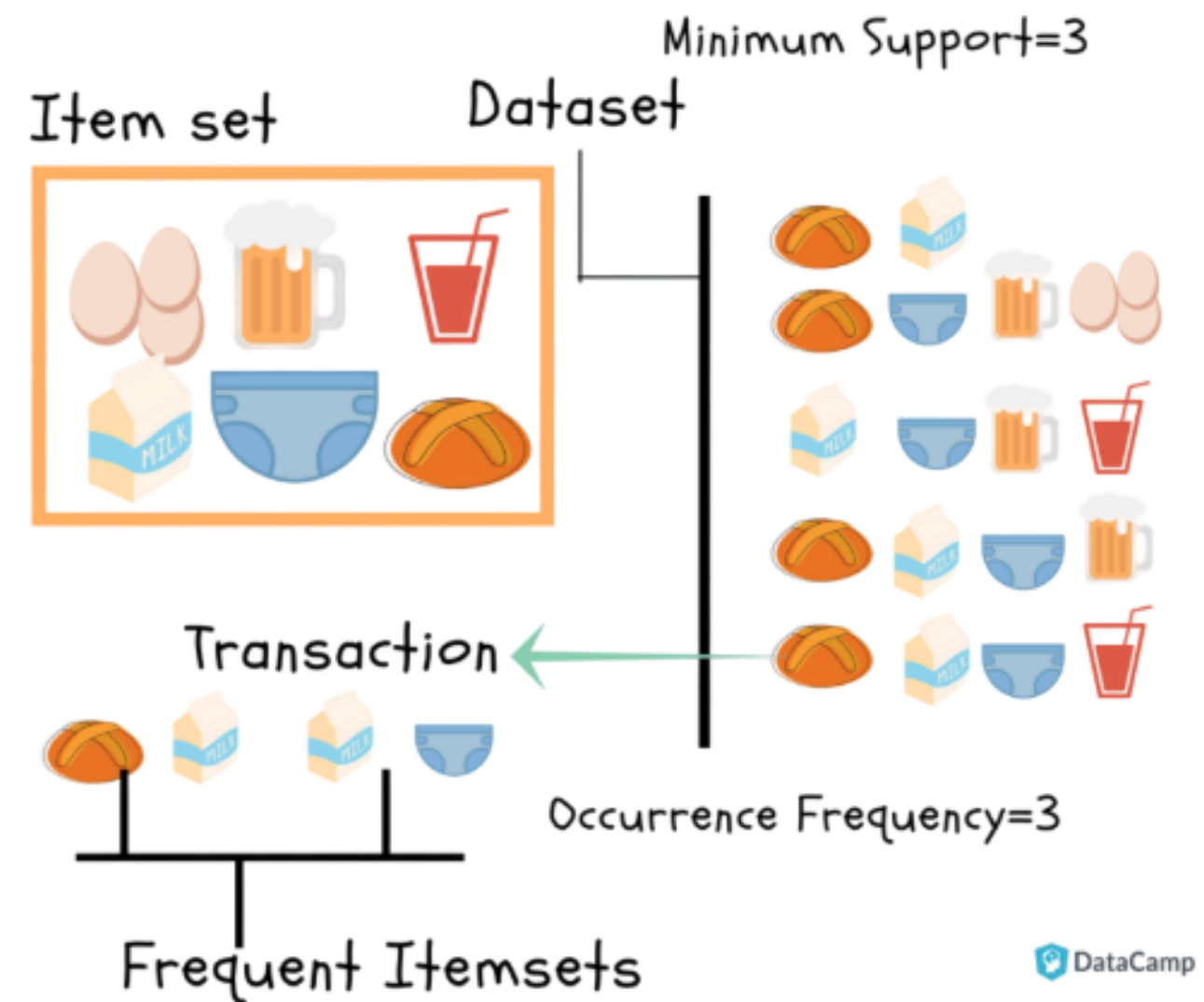
REGLAS DE ASOCIACION

¿Qué son las reglas de asociación?

A diferencia de la clasificación, las reglas de asociación pueden **predecir múltiples atributos a la vez**.

- El algoritmo busca combinaciones de datos (ítems) que aparezcan juntos con mucha frecuencia (alta cobertura).
- Descarta cualquier combinación que no alcance un número mínimo de apariciones.

Es un método de descarte progresivo.



Proceso paso a paso.

Nivel 1:

El algoritmo cuenta cuántas veces aparece cada cosa por separado.

Nivel 2:

Empieza a emparejar los ítems del Nivel 1.

Niveles 3 y 4:

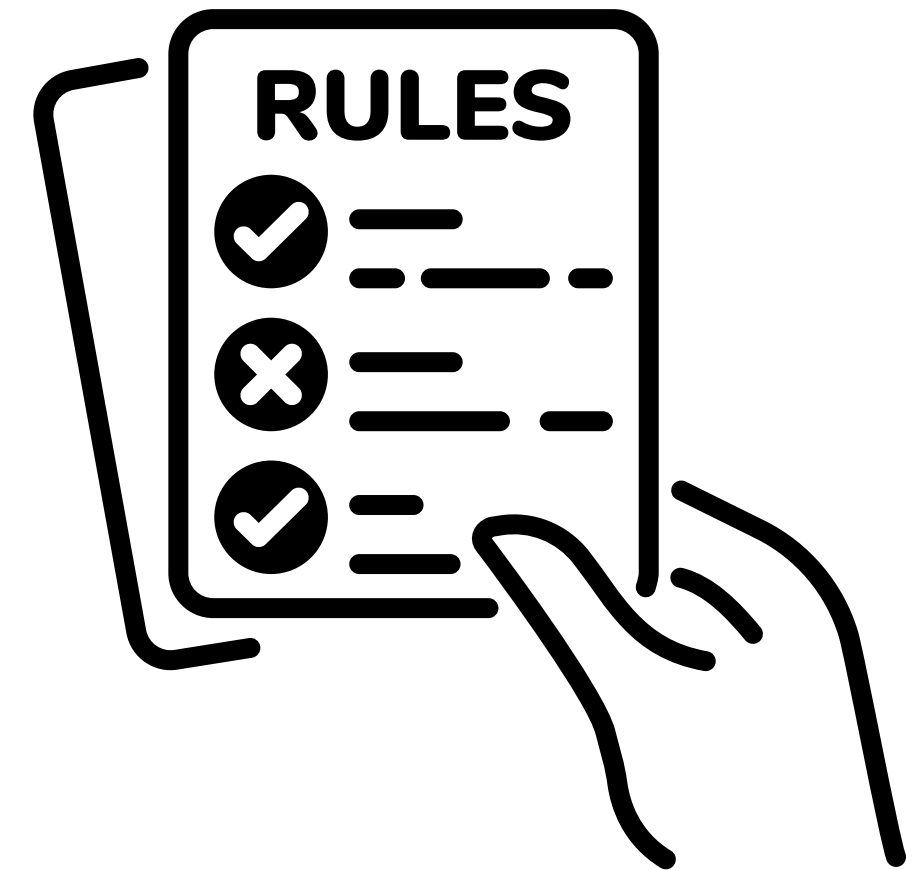
Toma las parejas exitosas del Nivel 2 y les agrega un tercer elemento

Fin:

El proceso se detiene cuando ya no puede armar conjuntos más grandes que cumplan con la regla de aparecer al menos 2 veces.

De Conjuntos a Reglas Lógicas

- Una vez obtenidos los "conjuntos de ítems" (los que pasaron el filtro de frecuencia), el algoritmo genera todas las combinaciones posibles de tipo "Si [A], entonces [B]".
- Múltiples posibilidades: Un solo conjunto de datos puede generar varias reglas.



Validacion.

Para asegurar que una regla es válida, el algoritmo aplica un filtro de Precisión (Confianza):

1. **Cálculo de Éxito:** Se divide el número de veces que la regla completa es correcta entre el número de veces que solo se cumple la primera parte.
2. **Umbral de Calidad:** Se establece un mínimo (ej. 100%). Si la regla falla aunque sea una vez en los datos de entrenamiento, se descarta.
3. **Resultado Final:** Solo las reglas que superan este nivel de confianza forman parte del modelo final.

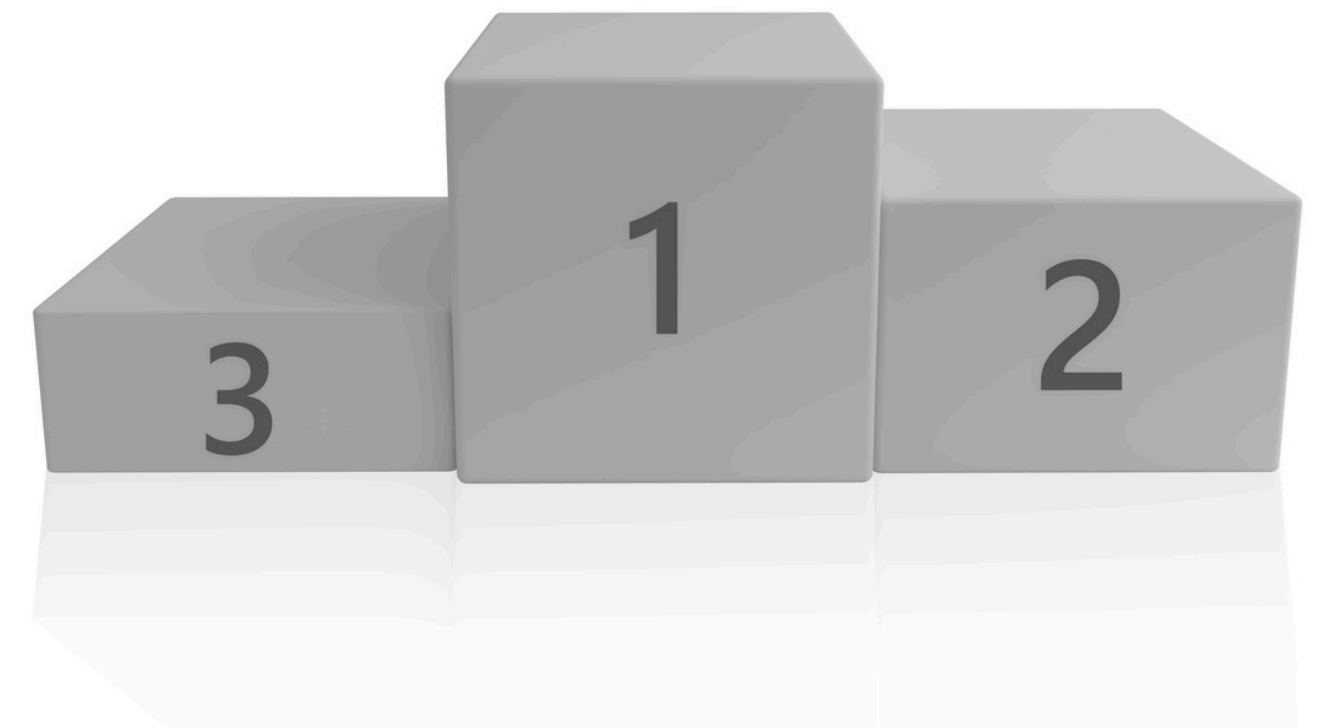


Selección de las mejores reglas

Después de generar muchas reglas de asociación, el sistema **no utiliza todas**, ya que muchas pueden ser irrelevantes o poco confiables. Por ello, **se realiza un proceso de selección** para **conservar únicamente las reglas más útiles**.

Las “best rules” son aquellas que:

- Tienen alta precisión
- Ocurren con suficiente frecuencia
- Representan patrones confiables en los datos



TSU

GRACIAS

P O R V E R

